

Projet Final de Développement Mobile

Gestionnaire d'Équipe de Football

Module : Développement Mobile avec React Native

Filière : 4IIR

Encadrant : Pr. Naji

Membres du groupe :

Abdelkhaleq Ibrahim

Louraoui Ayoub

Année Universitaire 2025–2026

Table des matières

1	Introduction et description du projet	3
1.1	Contexte du projet	3
1.2	Objectifs du projet	3
1.3	Public cible	3
2	Fonctionnalités principales et architecture	3
2.1	Fonctionnalités implémentées	3
2.1.1	Fonctionnalités communes	3
2.1.2	Fonctionnalités joueurs	3
2.1.3	Fonctionnalités managers	4
2.1.4	Fonctionnalités administrateurs	4
2.2	Architecture technique	4
2.2.1	Structure du projet	4
2.2.2	Flux d'authentification	4
2.2.3	Architecture de données	5
3	Technologies utilisées	5
3.1	Stack technique principale	5
3.2	Bibliothèques complémentaires	5
3.3	Outils de développement	5
4	Présentation fonctionnelle de l'application	5
4.1	Écrans d'authentification	5
4.2	Tableau de bord administrateur	6
4.3	Gestion des joueurs	6
4.4	Créateur de composition	6
4.5	Calendrier des matchs	6
5	Difficultés rencontrées et solutions proposées	6
5.1	Gestion du mode hors-ligne	6
5.1.1	Problème	6
5.1.2	Solution	6
5.2	Gestion des rôles utilisateurs	7
5.2.1	Problème	7
5.2.2	Solution	7
5.3	Performances des listes	7
5.3.1	Solution	7

6	Conclusion et perspectives	7
6.1	Bilan du projet	7
6.2	Perspectives	7

1 Introduction et description du projet

1.1 Contexte du projet

Dans le cadre du module **Développement Mobile avec React Native**, nous avons conçu et développé une application mobile complète dédiée à la gestion d’une équipe de football amateur. Cette application vise à répondre aux besoins organisationnels des clubs locaux en centralisant la gestion des joueurs, des matchs, des entraînements et des compositions tactiques dans une seule plateforme mobile.

1.2 Objectifs du projet

L’objectif principal de ce projet est de mettre en pratique l’ensemble des compétences acquises durant le module, notamment :

- le développement d’applications mobiles cross-platform avec React Native et Expo ;
- l’implémentation d’un système d’authentification sécurisé avec Firebase ;
- la gestion des données locales et distantes (SQLite et Firestore) ;
- la conception d’interfaces utilisateur ergonomiques et responsives ;
- le travail collaboratif à l’aide d’outils de gestion de projet et de versionnement.

1.3 Public cible

L’application est destinée à trois catégories principales d’utilisateurs :

- **Joueurs** : consultation du calendrier, des informations personnelles et déclaration de présence ;
- **Managers** : gestion de l’effectif, planification et création de compositions ;
- **Administrateurs** : supervision globale, statistiques et gestion des utilisateurs.

2 Fonctionnalités principales et architecture

2.1 Fonctionnalités implémentées

2.1.1 Fonctionnalités communes

- Authentification sécurisée avec gestion des rôles ;
- Interface utilisateur responsive et cohérente ;
- Notifications liées aux événements importants (matchs, entraînements).

2.1.2 Fonctionnalités joueurs

- Consultation de la liste des joueurs ;
- Accès au calendrier des matchs et résultats ;

- Déclaration et suivi de présence aux entraînements ;
- Gestion du profil utilisateur.

2.1.3 Fonctionnalités managers

- Gestion complète de l'effectif (ajout, modification, suppression) ;
- Planification des matchs et entraînements ;
- Création de compositions tactiques interactives ;
- Accès à un tableau de bord avec statistiques.

2.1.4 Fonctionnalités administrateurs

- Tableau de bord analytique global ;
- Gestion des utilisateurs et des permissions ;
- Consultation des journaux système ;
- Supervision de l'ensemble des données.

2.2 Architecture technique

2.2.1 Structure du projet

```
1 football-team-manager/  
2     src/  
3         components/  
4         screens/  
5         services/  
6         navigation/  
7         context/  
8         utils/  
9         storage/  
10    assets/  
11    App.js
```

Code 1 – Structure des dossiers du projet

2.2.2 Flux d'authentification

Le flux d'authentification débute par un écran de connexion ou d'inscription. Après validation des identifiants via Firebase Authentication, l'utilisateur est automatiquement redirigé vers une interface spécifique à son rôle (joueur, manager ou administrateur). Cette séparation garantit la sécurité des données et l'accès restreint aux fonctionnalités sensibles.

2.2.3 Architecture de données

L'architecture de données repose sur une approche hybride combinant Firebase Firestore pour le stockage cloud et SQLite pour la persistance locale. Cette solution permet un fonctionnement hors-ligne de l'application avec une synchronisation automatique des données lors du rétablissement de la connexion Internet.

3 Technologies utilisées

3.1 Stack technique principale

Technologie	Version	Rôle
React Native	0.73.6	Développement mobile cross-platform
Expo	SDK 50	Environnement de développement
Firebase	10.5.0	Authentification et base de données cloud
SQLite	13.4.0	Base de données locale hors-ligne
React Navigation	6.x	Navigation entre écrans
React Native Reanimated	3.9.0	Animations performantes

TABLE 1 – Technologies principales utilisées

3.2 Bibliothèques complémentaires

- @react-native-community/datetimestpicker ;
- react-native-chart-kit ;
- expo-notifications ;
- @expo/vector-icons ;
- react-native-dotenv.

3.3 Outils de développement

- Taiga.io pour la gestion de projet agile ;
- Git et GitHub pour le versionnement ;
- Expo Go pour les tests sur appareils physiques ;
- Firebase Console pour l'administration des données.

4 Présentation fonctionnelle de l'application

4.1 Écrans d'authentification

Les écrans d'authentification comprennent un écran de connexion, un écran d'inscription ainsi qu'un écran de profil utilisateur. L'interface est volontairement simple et

intuitive afin de faciliter l'accès à l'application pour tous les profils d'utilisateurs.

4.2 Tableau de bord administrateur

Le tableau de bord administrateur fournit une vue synthétique de l'activité globale de l'application à travers des indicateurs et statistiques permettant une prise de décision rapide.

4.3 Gestion des joueurs

Ce module permet la consultation de l'effectif et la gestion complète des joueurs via des formulaires dédiés accessibles aux managers.

4.4 Créateur de composition

Le créateur de composition permet aux managers de positionner les joueurs sur un terrain virtuel afin de définir les formations tactiques avant les matchs.

4.5 Calendrier des matchs

Le calendrier affiche les matchs passés et à venir avec leurs informations essentielles telles que la date, l'adversaire et le résultat.

5 Difficultés rencontrées et solutions proposées

5.1 Gestion du mode hors-ligne

5.1.1 Problème

Synchronisation des données entre Firebase et SQLite tout en garantissant la cohérence lors des reconnections.

5.1.2 Solution

```
1 export const syncOfflineData = async () => {  
2   const unsyncedPlayers = await getUnsyncedPlayers();  
3   for (const player of unsyncedPlayers) {  
4     await addPlayerToFirestore(player);  
5     markPlayerAsSynced(player.id);  
6   }  
7 };
```

Code 2 – Synchronisation SQLite-Firebase

5.2 Gestion des rôles utilisateurs

5.2.1 Problème

Séparation des permissions et navigation conditionnelle.

5.2.2 Solution

```
1 switch (role) {  
2   case 'admin': return <AdminNavigator />;  
3   case 'manager': return <ManagerNavigator />;  
4   case 'player': return <PlayerNavigator />;  
5 }
```

Code 3 – Gestion des rôles

5.3 Performances des listes

5.3.1 Solution

```
1 <FlatList  
2   data={players}  
3   initialNumToRender={10}  
4   windowSize={5}  
5 />
```

Code 4 – Optimisation avec FlatList

6 Conclusion et perspectives

6.1 Bilan du projet

Le projet **Gestionnaire d'Équipe de Football** a abouti à une application fonctionnelle, modulaire et évolutive répondant à un besoin réel.

6.2 Perspectives

- intégration d'API sportives ;
- chat en temps réel ;
- support multilingue ;
- publication sur les stores.

Annexes

Annexe A : Configuration Firebase

```
1 const firebaseConfig = {  
2   apiKey: FIREBASE_API_KEY,  
3   authDomain: FIREBASE_AUTH_DOMAIN,  
4   projectId: FIREBASE_PROJECT_ID  
5 };
```

Annexe B : Structure Firestore

Collection	Description
users	Utilisateurs avec rôles
players	Effectif de l'équipe
matches	Calendrier des matchs
logs	Journaux système

Annexe C : Liens

- [GitHub](#)
- [Taiga](#)
- [Firebase Console](#)